GEMAMX v2

Pipeline de Validación Farmacológica Computacional

Fases M3 a M5:

IA Estructural · Docking Molecular · Insights Clínicos

Punto de partida	Archivos de salida consolidados Fases 1–3 (v2/data/fasta/*.fasta + fasta_sequences.json)
Scripts M3	`predict_structures_romero_boltz.py` / `calc_structural_metrics_romero.py`
Scripts M4	`prepare_docking_romero.py` / `run_docking_romero.py` / `build_docking_complexes_romero.py`
Script M5	`build_variant_clinical_summary.py`

10. M3 — IA Estructural: Predicción con Boltz-1

Las secuencias FASTA generadas en la Fase 3 son el insumo directo de la predicción de estructuras tridimensionales. El pipeline utiliza Boltz-1, un modelo de predicción de estructura proteica basado en aprendizaje profundo, para generar coordenadas 3D de cada variante WT y MUT. Esta fase es la más costosa computacionalmente y por eso se ejecuta en GPU en Google Colab.

10.1 Lógica de Decisión: Secuencia Completa vs Fragmento

El script **`predict_structures_romero_boltz.py`** implementa una lógica crítica de bifurcación según el tamaño de la proteína. La constante clave es `BOLTZ_LIMIT = 390` aminoácidos: proteínas por debajo de este umbral se procesan en secuencia completa; las que lo superan se recortan a un fragmento centrado en la posición de la mutación con ventana de ±180 aa (`FRAGMENT_WINDOW = 180`):

```
BOLTZ_LIMIT = 390     # aa — umbral de longitud completa
FRAGMENT_WINDOW = 180  # aa a cada lado de la posición mutada

def get_fragment(seq, mut_pos, window=FRAGMENT_WINDOW):
    start = max(0, mut_pos - window - 1)   # 0-based
    end   = min(len(seq), mut_pos + window)
    return seq[start:end], start+1, end    # retorna (secuencia, start_1based, end_1based)

short_protein = len(wt_seq) <= BOLTZ_LIMIT and len(mut_seq) <= BOLTZ_LIMIT
if short_protein:
    mode = 'full'       # TP53 (393 aa), GATA3 (444 — ligero!), PTEN (403)
else:
    mode = 'fragment'   # PIK3CA (1068 aa), ERBB2 (1255 aa), MAP3K1 (1512 aa)
```

Gen	Long. total (aa)	Modo	Fragmento predicho	Justificación
TP53	393	full	1–393	Cabe completa: pLDDT uniforme en toda la cadena.
PTEN	403	full	1–403	Justo bajo el límite; se predice íntegra.
GATA3	444	full*	1–444	Supera límite pero jobs muestran mode=full en algunas variantes.
AKT1	480	fragment	1–197 (E17K)	Mutación en N-terminal; fragmento cubre dominio PH completo.
PIK3CA	1,068	fragment	867–1068 (H1047R)	Dominio kinasa; fragment de 202 aa centrado en pos 1047.
ERBB2	1,255	fragment	230–430 (S310F)	Dominio extracelular; fragment de 200 aa.
MAP3K1	1,512	fragment	±180 aa	La proteína más larga del dataset; fragmentación necesaria.

10.2 Gestión de Jobs: Caché y Deduplicación de WT

Una optimización fundamental del script es que el WT de un gen se predice una sola vez aunque haya múltiples variantes del mismo gen. Esto se controla con el set `generated_wt` y el argumento `--only-missing`:

```
generated_wt = set()
for job in jobs:
    # WT: solo se predice si no se ha procesado aún
    if job['wt_key'] not in generated_wt:
        if not existing_structure_path(job['wt_key']) or args.force:
            ok, result = run_boltz(job['wt_key'], job['wt_input_seq'])
        generated_wt.add(job['wt_key'])   # marcado independiente del éxito
    # MUT: siempre se predice (es única por variante)
    ok, result = run_boltz(job['mut_key'], job['mut_input_seq'])
```

La convención de nombres distingue WT de MUT: el WT se nombra `WT__{GEN}_{VARIANTE}` (ej. `WT__PIK3CA_H1047R.pdb`) y el MUT simplemente como `PIK3CA_H1047R.pdb`. Esto permite comparación directa de archivos.

10.3 Ejecución de Boltz-1 y Recuperación de Salidas

El pipeline invoca Boltz a través de su CLI. Para cada variante crea un directorio de run independiente, escribe el FASTA de entrada, ejecuta la predicción y luego busca el archivo CIF resultante en múltiples rutas candidatas (la estructura del directorio de salida de Boltz varía entre versiones):

```
cmd = ['boltz', 'predict', str(fasta_path),
       '--out_dir', str(run_dir), '--output_format', 'mmcif']
subprocess.run(cmd, cwd=ROOT)

# Búsqueda de salida en rutas candidatas (robusto a cambios de versión):
exact_candidates = [
    run_dir / name / 'predictions' / name / f'{name}_model_0.cif',
```

```
    run_dir / 'predictions' / name / f'{name}_model_0.cif',
    run_dir / f'{name}_model_0.cif',
]
# Fallback: buscar recursivamente el .cif más grande
files = sorted(run_dir.rglob('*.cif'), key=lambda p: p.stat().st_size, reverse=True)
```

Las estructuras resultantes se copian a `v2/data/structures/` y los archivos de confianza JSON de Boltz se mueven a `v2/data/structures/meta/`. Los índices `manifest_structures.json` y `structure_jobs.json` se actualizan incrementalmente en cada corrida, permitiendo reanudar sin rehacer trabajo completado.

10.4 Cálculo de Métricas Estructurales (calc_structural_metrics_romero.py)

Una vez en disco las estructuras PDB/CIF, el script **`calc_structural_metrics_romero.py`** compara sistemáticamente cada par WT/MUT usando dos métricas: pLDDT como proxy de calidad y RMSD como medida de cambio conformacional.

10.4.1 Extracción de átomos Cα y pLDDT

El parser lee el PDB línea a línea buscando registros `ATOM` con nombre `CA` (carbono alfa). El campo `B-factor` (columnas 60-66) se usa como proxy del score pLDDT reportado por Boltz, ya que este modelo escribe la confianza en ese campo. Una condición de escala detecta si los valores están en rango 0-1 (los escala ×100) o ya en 0-100:

```
def parse_pdb_ca(path):
    for line in f:
        if not line.startswith('ATOM'): continue
        if line[12:16].strip() != 'CA': continue
        resseq = int(line[22:26])
        x,y,z  = float(line[30:38]), float(line[38:46]), float(line[46:54])
        b      = float(line[60:66])   # B-factor = pLDDT
        atoms.append({'resseq':resseq,'coord':(x,y,z),'plddt':b})

def scale_plddt(value):
    if value <= 1.5: return value * 100.0   # escala 0-1 → 0-100
    return value                             # ya está en 0-100
```

10.4.2 RMSD Global y RMSD en Ventana Activa

Se calculan dos RMSDs distintos. El **RMSD global** compara todos los Cα alineados en su orden de resseq. El **RMSD activo** opera sobre una ventana de ±12 aa alrededor de la posición mutada, capturando cambios locales en el sitio de interacción con el fármaco:

```
LOCAL_WINDOW = 12  # residuos a cada lado de la mutación

# Ventana local: coordenadas relativas al fragmento
local_start = max(frag_start, mut_pos - LOCAL_WINDOW)
local_end   = min(frag_end,   mut_pos + LOCAL_WINDOW)

# Mapeo resseq → posición genómica del fragmento
genomic_like = frag_start + (resseq - 1)

def rmsd(coords_a, coords_b):
    n = min(len(coords_a), len(coords_b))
    s = sum((ax-bx)**2+(ay-by)**2+(az-bz)**2
            for (ax,ay,az),(bx,by,bz) in zip(coords_a[:n],coords_b[:n]))
    return math.sqrt(s/n)
```

> **Significado biofísico de los valores RMSD:**

• RMSD global < 1.0 Å: la mutación no perturba la arquitectura global de la proteína. Buen predictor de que el fármaco puede unirse sin requerir conformación alternativa.

• RMSD activo < 1.5 Å (color amber en el output): cambio local moderado — el bolsillo puede estar ligera-mente alterado pero es accesible.

• RMSD activo ≥ 2.0 Å (color rojo en el output): reorganización conformacional significativa del sitio activo — mayor incertidumbre en el docking subsecuente.

Variante	pLDDT WT	pLDDT MUT	RMSD global (Å)	RMSD activo (Å)	Fragmento predicho
PIK3CA_H1047R	84.8	85.3	0.78	0.91	867–1068 (202 aa)
AKT1_E17K	85.4	85.2	0.94	0.67	1–197 (197 aa)
PIK3CA_E542K	90.2	90.2	1.50	1.80 ⚠	362–722 (361 aa)
ERBB2_S310F	86.1	86.4	0.65	0.73	230–430 (200 aa)
TP53_R175H	78.5	77.9	1.12	1.35	full (393 aa)
PTEN_R173H	81.2	80.8	0.95	1.10	full (403 aa)

11. M4 — Docking Molecular: Preparación de Assets

La fase de docking es la más voluminosa en términos de archivos generados: **902 pares (log + pdbqt)** cubriendo 41 receptores × 11 fármacos. Se divide en tres sub-fases ejecutadas por scripts independientes.

11.1 Preparación de Ligandos (prepare_docking_romero.py — Bloque 1/2)

El catálogo de 11 fármacos se define explícitamente con su **ChEMBL ID**, diana principal y estatus regulatorio. Los SDF se descargan desde la API de ChEMBL y se convierten a PDBQT usando **RDKit + Meeko**:

Fármaco	ChEMBL ID	Diana	Estatus	Conversión ligando
Alpelisib	CHEMBL3301610	PI3Kα	**FDA**	SDF → AddHs → ETKDG → MMFF → PDBQT
Capivasertib	CHEMBL3301612	AKT1/2/3	**FDA**	SDF → RDKit → Meeko
Palbociclib	CHEMBL1771552	CDK4/6	**FDA**	SDF → RDKit → Meeko
Abemaciclib	CHEMBL3545110	CDK4/6	**FDA**	SDF → RDKit → Meeko
Ribociclib	CHEMBL3545283	CDK4/6	**FDA**	SDF → RDKit → Meeko

Copanlisib	CHEMBL3137587	Pan-PI3K	**FDA**	SDF → RDKit → Meeko
Everolimus	CHEMBL1560600	mTOR	**FDA**	SDF → RDKit → Meeko
Pictilisib	CHEMBL2178688	Pan-PI3K	Trial	SDF → RDKit → Meeko
Gedatolisib	CHEMBL3188482	PI3K/mTOR	Trial	SDF → RDKit → Meeko
Taselisib	CHEMBL3301609	PI3Kα	Trial	SDF → RDKit → Meeko
Buparlisib	CHEMBL2180717	Pan-PI3K	Trial	SDF → RDKit → Meeko

La conversión de ligandos usa el pipeline RDKit+Meeko: selección del fragmento más grande (`LargestFragmentChooser`), adición de hidrógenos, embebido 3D con `ETKDG`, minimización de energía con campo de fuerzas `MMFF`, y finalmente preparación PDBQT con Meeko. Esto garantiza geometría 3D correcta con cargas Gasteiger implícitas en el formato AutoDock.

11.2 Preparación de Receptores (prepare_docking_romero.py — Bloque 2/2)

Los receptores se convierten de PDB/CIF a PDBQT en dos pasos. Primero **OpenBabel** limpia la estructura (elimina aguas, añade hidrógenos a pH 7.4, regenera el PDB canónico). Luego un parser PDB propio convierte línea a línea a formato PDBQT:

```
# Paso 1: OpenBabel — limpieza y protonación
subprocess.run([OBABEL_EXE, str(struct_path),
    '-O', str(tmp_pdb), '-xr', '--addh', '-p', '7.4'], timeout=120)

# Paso 2: Conversión PDB → PDBQT
# Asignación de tipo AutoDock4 por elemento:
element = line[76:78].strip().upper()
ad_type = 'HD' if element == 'H' else element[:2]

# Formato de línea PDBQT (columnas fijas FORTRAN):
# ATOM  serial name resname chain resseq   x       y       z    occ temp  charge type
```

Los archivos `{VARIANTE}_clean.pdb` y `{VARIANTE}.pdbqt` se generan en `v2/data/docking/receptors/`. El primero se reutiliza en la Fase M5 para construir complejos visualizables. El pipeline procesó los 41 receptores con éxito, confirmado en `prepared_receptors.json`.

11.3 Ejecución de Docking (run_docking_romero.py)

11.3.1 Cálculo Automático del Centro de la Caja de Búsqueda

AutoDock Vina requiere definir una caja cúbica de búsqueda (search box) centrada en la región de interés del receptor. En lugar de definir coordenadas manuales, el pipeline calcula automáticamente el **centro de masa** del receptor completo como aproximación del centroide de unión:

```
def center_of_mass(pdbqt_path):
    xs,ys,zs = [],[],[]
    for line in open(pdbqt_path):
        if line.startswith(('ATOM','HETATM')):
            xs.append(float(line[30:38]))
            ys.append(float(line[38:46]))
            zs.append(float(line[46:54]))
```

```
    n = len(xs)
    return sum(xs)/n, sum(ys)/n, sum(zs)/n

# Caja fija ±20 Å (25 Å total en cada dimensión)
cmd = [vina_exe, '--size_x','25','--size_y','25','--size_z','25',
       '--center_x',f'{cx:.3f}', ...]
```

11.3.2 Ejecución y Parsing del Score de Afinidad

Por cada par (receptor, ligando) se ejecuta Vina con `exhaustiveness=8` y `num_modes=5`. El log de Vina escribe una tabla de modos con el formato `'    1   -7.732   ...'`. El script extrae solo el score del Modo 1 (mejor pose) mediante expresiones regulares con tres patrones de fallback para compatibilidad entre versiones de Vina:

```
patterns = [
    r'^\s+1\s+([-\d]+\.\d+)',      # Vina 1.2.x estándar
    r'^\s+1\s+\|\s+([-\d]+\.\d+)', # Vina con separador |
    r'^   1\s+([-\d]+\.\d+)',        # formato compacto
]
# Solo acepta valores entre -20 y 0 kcal/mol (rango físicamente válido)
if -20 < val < 0: return val
```

El mecanismo de caché es importante para la eficiencia: si el `log_file` ya existe y contiene un score válido, el par se salta sin relanzar Vina. Esto permite reanudar corridas parciales sin recomputar los pares ya completados.

Los resultados se escriben incrementalmente a `vina_affinities.json` después de cada receptor, y al finalizar la corrida completa se actualiza también `structural_metrics.json` con los campos `best_vina_drug` y `best_vina_score` para cada variante.

11.4 Construcción de Complejos Receptor-Ligando (build_docking_complexes_romero.py)

Este script fusiona el PDB limpio del receptor con la mejor pose del ligando (extraída del `*_out.pdbqt`) para generar complejos visualizables con PyMOL o Blender. El algoritmo opera en tres etapas:

- Extrae MODEL 1 (mejor pose) del PDBQT de salida de Vina. Si el PDBQT no tiene marcadores MODEL/ENDMDL, asume que todo el contenido es la primera pose.
- Lee el receptor clean.pdb línea a línea, renumerando átomos secuencialmente.
- Convierte cada línea ATOM/HETATM del ligando PDBQT a formato PDB estándar, añadiendo chain L, resname LIG, y deduciendo el elemento químico del tipo AutoDock4 (HD→H, CL→Cl, etc.).
- Escribe el complejo con cabecera HEADER, TITLE, REMARK con el score de afinidad, receptor, TER, ligando, TER, END.

```
# Header del complejo:
'HEADER    GEMAMX V2 DOCKING COMPLEX'
f'TITLE     {variant_id} WITH {drug_safe}'
f'REMARK   2 BEST_VINA_AFFINITY {affinity:.3f} KCAL/MOL'

# Parsing del score desde REMARK en el pdbqt de Vina:
if line.startswith('REMARK VINA RESULT:'):
    for token in line.split():
        if -20.0 < float(token) < 0.0: return float(token)
```

El índice de complejos `docking/complex/index.json` registra la ruta del complejo, el nombre del fármaco y la afinidad de Vina para cada par variante-fármaco, habilitando navegación programática desde la interfaz M5.

12. M4 — Resultados Consolidados de Docking

La corrida completa produjo scores de afinidad para **451 pares únicos válidos** (41 receptores × 11 fármacos, con 1 null en PTEN_I28F+Palbociclib). Los hallazgos más relevantes por gen:

12.1 Ranking Global — Top Scores por Tier

Tier	Variante	Mejor fármaco	Score (kcal/mol)	2° fármaco	Observación
S	GATA3_Q83L	Pictilisib	**-10.22**	Alpelisib -10.02	Mejor score absoluto del dataset
S	TP53_D57N	Pictilisib	**-10.20**	Alpelisib -9.549	Todos los 11 fármacos < −5.3
S	PTEN_D297H	Pictilisib	-9.455	Palbociclib -9.283	Palbociclib también supera −9.0
S	PIK3CA_E545G	Pictilisib	-9.417	Taselisib -8.978	E545G más potente que E545K
S	AKT1_L52R	Alpelisib	-9.086	Pictilisib -9.069	Único caso donde Alpelisib supera Pictilisib
A	PIK3CA_E453K	Pictilisib	-8.912	Copanlisib -8.693	Mayor RMSD local: bolsillo activo distorsionado
A	AKT1_E17K	Pictilisib	-8.345	Alpelisib -7.967	Variante más frecuente del cohorte con alta afinidad
⚠	PIK3CA_M1004I	Buparlisib	-8.670	Palbociclib -2.293	ANOMALÍA: delta 6.4 kcal/mol — Palbociclib no convergió

Dominio de Pictilisib:

Pictilisib obtiene el mejor score en 29 de los 41 receptores (70.7%). Alpelisib gana en 7, Palbociclib en 2, Abemaciclib en 2. Score promedio de Pictilisib: -7.8 kcal/mol, el mejor del panel.

13. M5 — Insights Clínicos: Contextualización del Cohorte

La fase M5 cruza los resultados computacionales con los metadatos clínicos del cohorte Romero para generar **variant_clinical_summary.json**, el archivo de 61.67 KB que integra frecuencia de variante, perfil de afinidad farmacológica y características de los pacientes portadores.

13.1 Fuentes de Datos para el Cruce Clínico

El script **build_variant_clinical_summary.py** opera sobre tres archivos:

- raw_mutations.json: 28,114 mutaciones — proporciona el mapeo sample_id → variante.
- clinical_metadata.json: 130.94 KB de metadatos clínicos del cohorte (edad, subtipo IHC, PAM50, grado SBR, histología).
- top_variants.json: define qué variantes se incluyen en el análisis (las 49 priorizadas).

```
# Construcción del índice clínico por muestra
clinical_by_sample = {row['sample_id']: row for row in clinical_rows}

# Identificación de muestras portadoras de cada variante
for mutation in raw_mutations:
    variant_id = f"{mutation['hugo_symbol']}_{mutation['protein_change']}"
    if variant_id in target_variants:
        samples_by_variant[variant_id].add(mutation['sample_id'])

# Cruce: para cada muestra portadora, extraer su fila clínica
matched_rows = [clinical_by_sample[sid] for sid in sample_ids
                if sid in clinical_by_sample]
```

13.2 Campos Clínicos Analizados

Campo clínico	Tipo de análisis	Descripción
age	Media, min, max	Edad al diagnóstico de los pacientes portadores de la variante.
age_class	Counter + top	Clasificación Older/Younger basada en corte de edad del estudio Romero.
ihc_subtype	Counter + top	Subtipo IHC: Luminal A, Luminal B, HER2-enriched, Triple Negativo.
class_ihc	Counter + top	Clasificación clínica práctica: HR+/HER2−, HR+/HER2+, HER2+, TNBC.
pam50	Counter + top	Subtipo molecular PAM50: LumA, LumB, HER2, Basal, Normal-like.
grade_sbr	Counter + top	Grado histológico Scarff-Bloom-Richardson: I, II, III.
histology	Counter + top	Histología: IDC (carcinoma ductal invasivo), IL (lobulillar), Other.

13.3 Ejemplo de Output Clínico — PIK3CA H1047R

La variante más frecuente del cohorte (15 casos, 11.19%) muestra un perfil clínico consistente con la literatura de cáncer de mama hormono-positivo:

```
{
  "variant_id": "PIK3CA_H1047R",
  "sample_count": 15,
  "matched_clinical_count": 15,
  "age": { "mean": 56.71, "min": 39, "max": 92 },
  "age_class_distribution": { "Older": 11, "Younger": 3 },
  "ihc_subtype_distribution": { "Luminal A": 12, "Luminal B": 2, "Her2-enriched": 1 },
  "class_ihc_distribution": { "HR+/HER2-": 12, "HR+/HER2+": 2, "HER2+": 1 },
  "pam50_distribution": { "LumB": 2, "LumA": 5 },
  "grade_distribution": { "II": 10, "I": 2, "III": 1 },
  "histology_distribution": { "IDC": 12, "IL": 1, "Other": 1 },
  "highlights": {
    "top_age_class": "Older",
    "top_ihc_subtype": "Luminal A",
    "top_class_ihc": "HR+/HER2-",
    "top_pam50": "LumA"
  }
}
```


Interpretación clínica: el 80% de los pacientes con PIK3CA H1047R son HR+/HER2−, con predominio de subtipo Luminal A y grado II. Esto es biológicamente consistente: PIK3CA H1047R activa constitutivamente el eje PI3K/AKT, un mecanismo de escape a la terapia endocrina en tumores luminales. El score de docking de Pictilisib (-7.73 kcal/mol) y Alpelisib (-7.73 kcal/mol) refuerza la relevancia terapéutica de los inhibidores PI3K en esta subpoblación.

14. Flujo de Datos Completo M3–M5

Cada fase produce archivos que son entrada directa de la siguiente. El pipeline es reproducible y reanudable en cualquier punto:

Fase	Script	Entrada clave	Salidas principales
M3a	predict_structures_romero_boltz.py	fasta_sequences.json	v2/data/structures/WT__*.pdb / {VAR}.pdb (87 PDB) | manifest_structures.json | structure_jobs.json
M3b	calc_structural_metrics_romero.py	structure_jobs.json	structural_metrics.json (pLDDT WT/MUT, RMSD global/activo por variante)
M4a	prepare_docking_romero.py	manifest_structures.json	docking/ligands/*.pdbqt (22 archivos) | docking/receptors/*_clean.pdb + *.pdbqt (82 archivos) | prepared_receptors.json
M4b	run_docking_romero.py	receptors/*.pdbqt, ligands/*.pdbqt	docking/results/*_log.txt + *_out.pdbqt (902 archivos) | vina_affinities.json | structural_metrics.json (actualizado)
M4c	build_docking_complexes_romero.py	*_clean.pdb + *_out.pdbqt	docking/complex/*_complex.pdb | docking/complex/index.json

M5	build_variant_clinical_summary.py	raw_mutations.json, clinical_metadata.json, top_variants.json	variant_clinical_summary.json (61.67 KB — 49 variantes con perfil clínico completo)

15. Inventario Final de Archivos del Pipeline

Archivo / Directorio	Tamaño	Contenido
v2/data/structures/ (87 PDB)	16.77 MB	44 MUT + 43 WT en formato PDB. Fragmentos Boltz-1 por variante.
v2/data/manifest_structures.json	5.06 KB	Índice: variant_key → ruta relativa del PDB en disco.
v2/data/structure_jobs.json	24.43 KB	Metadatos por job: fragmento, posición mutada, rutas WT/MUT, modo.
v2/data/structural_metrics.json	18.79 KB	pLDDT WT/MUT, RMSD global/activo, best_vina_drug y vina_scores para cada variante.
v2/data/docking/ligands/ (22 archivos)	65.19 KB	11 SDF + 11 PDBQT. Un par por fármaco.
v2/data/docking/receptors/ (82 archivos)	44.11 MB	41 _clean.pdb + 41 .pdbqt. Un par por variante.
v2/data/docking/results/ (902 archivos)	8.42 MB	451 _log.txt + 451 _out.pdbqt. Resultados por par variante-fármaco.
v2/data/docking/vina_affinities.json	12.96 KB	Scores de afinidad: { variante → { fármaco → kcal/mol } }.
v2/data/docking/prepared_receptors.json	2.59 KB	Índice: variante → ruta del PDBQT receptor.
v2/data/docking/complex/index.json	—	Índice de complejos receptor-ligando: variante → fármaco → {ruta, afinidad}.
v2/data/variant_clinical_summary.json	61.67 KB	49 variantes con edad, subtipo IHC, PAM50, grado, histología y highlights.
v2/data/clinical_metadata.json	130.94 KB	Metadatos clínicos individuales de las 134 muestras del cohorte.